

From Abrupt to Gradual Drift in Federated Learning: Label-Free Reactive Retraining and a Service-Downtime Metric

Author Names

Institute of Computing, University of Campinas (UNICAMP), Brazil

TODO: emails

Abstract—Federated learning (FL) models deployed in production face concept drift: the data distribution shifts after deployment, and model quality degrades until a retraining action is taken. In real deployments the shift is often gradual: the fraction of corrupted inputs rises slowly over a transition interval whose duration is the drift width used in the drift-detection literature. We study reactive retraining in synchronous FL as the drift transition lengthens from abrupt to gradual. Each client monitors its unlabeled input stream with a local uncertainty-based detector (Monte Carlo dropout, Shannon entropy, and an ADWIN adaptive-window test); the server aggregates the local alarms over a short tick window and, at most once, triggers a single global retraining round executed on each client’s current mixture. The primary metric is the service downtime, the cumulative virtual time in which the served-model accuracy stays below a quality threshold. In 60 paired episodes (four calibrated corruptions, five seeds, three ramp durations of 50–200s) over a fixed 600s horizon, compared with a paired baseline that never retrains, detection succeeds in 59 of 60 episodes (14 of 15 in the Gaussian-noise scenario); the detection delay grows with the ramp duration for the three visual corruptions (98–140s) and reaches 256s for Gaussian noise; the reactive policy reduces the mean downtime from 408–562s in the baseline to 0–178s, reaching zero downtime at $\tau = 0.5$ in the longest-ramp fog scenario; and long ramps preserve clean-set accuracy for the visual corruptions (1–2 points loss instead of 9–21). Within the exercised regime (CIFAR-10, IID partition, ten clients, a single drift and at most one retraining round), a reactive label-free policy is effective and less destructive to previously learned knowledge.

Index Terms—federated learning, concept drift, drift detection, reactive retraining, unsupervised detection, service downtime.

I. Introduction

Federated learning (FL) trains a shared model from data that remains distributed across clients [1]. Once deployed, the model is served in an environment whose data distribution can change; such concept drift degrades the served model’s quality until a retraining action is taken, and detecting it—usually without access to labels at inference time—is a prerequisite for triggering that action. A common simplification in the literature is to inject drift abruptly: the distribution switches at a single instant. In practice, however, the shift is often gradual: new data appears mixed with old data, and the proportion of shifted inputs grows slowly over a transition interval

whose duration is the drift width used in the drift-detection literature [3], [18]. We model this continuum as a linear ramp of the corrupted-data fraction from zero to one over a configurable number of monitoring ticks [3]; Casado et al. [2] define gradual drift as the case in which the new concept appears mixed with the old one. Our grid spans short ramps (equivalent to the abrupt regime) and a long ramp in which the detector operates mid-transition, covering the boundary between the two regimes.

Our reactive policy follows the architecture that the FL drift literature converged to: local detection, central decision [2], [4]–[10], [23], [24]. Each client runs an uncertainty drift detector (UDD) [11]: Monte Carlo (MC) dropout [12] over T stochastic forwards, mean Shannon entropy, and an ADWIN adaptive window test [13] per client. The server collects the per-client alarms over a sliding window of two monitoring ticks and triggers when the fraction of distinct flagged clients reaches a quorum. The triggered action is a single global retraining round on the mixture each client currently serves, in line with the explicit retraining budgets of DriftGuard [8] and ShiftEx [26] and Modyn’s trigger/data-selection separation [20].

We evaluate the policy under an operational lens. The primary metric is the service downtime: the cumulative virtual time in which the corrupted accuracy of the served model stays below a quality threshold τ , computed as a stepwise integral without interpolation over a fixed horizon. To our knowledge, no work in the reviewed FL drift literature quantifies the time a served model stays below a quality threshold; we detail this gap in Section II-E.

Our contributions are: (i) a service-downtime metric for FL under drift, computed as a stepwise integral over a fixed horizon; (ii) a collective quorum policy aggregating per-client label-free alarms over a short tick window, a mechanism absent from the reviewed literature, closest to the participation gate of FL-MalDrift [7], assessed descriptively (ablations against alternative triggers are future work); (iii) a federated installation of UDD, with one detector per client sharing the global model, extending the centralized detector of Baier et al. [11]; (iv) an auditable experimental infrastructure that pairs the reactive arm with a no-retraining baseline from the same checkpoint,

random state, stream, and horizon, and validates the pair before publication; and (v) an empirical characterization of gradual drift over 60 paired episodes.

Our main findings are: (i) detection succeeds in 59 of 60 episodes (14 of 15 in the Gaussian-noise scenario); (ii) the detection delay grows with the ramp duration for the three visual corruptions, while Gaussian noise exhibits larger delays and dispersion; (iii) the reactive policy reduces the mean downtime from 408–562s in the baseline to 0–178s, reaching zero downtime in the longest-ramp fog scenario; and (iv) long ramps preserve clean-set accuracy for the visual corruptions, because the retraining set retains a substantial share of clean data.

II. Background and Related Work

A. Concept drift in federated learning

Concept drift is the change in the data distribution over time [19], [21]; the taxonomy distinguishes virtual drift (only $P(X)$ changes) from real drift (the conditional $P(Y | X)$ changes), and sudden, gradual, and recurrent drifts [19]. Staggered drifts, in which different clients drift at different times, are studied by Jothimurugesan et al. [4]. Gradual drift, the regime studied here, mixes the new concept with the old one with a growing proportion over time [2]; the transition duration is the drift width, the standard parameter controlling the speed of the change [3], [18].

Our scope is deliberately delimited to drifts accompanied by a change in $P(X)$ (covariate shift with preserved labels), the assumption under which label-free detection is structurally possible: label-free methods are deterministic functions of the inputs, so a change confined to $P(Y | X)$ produces no observable signal, a blind spot measured by Solozobov [17] and declared by Baier et al. [11].

B. Local detection, central decision

A recurring finding in FL drift research is that drift detection must be local. Jothimurugesan et al. [4] show that testing the aggregated error at the server fails during staggered transitions, because the errors of drifted clients and of clients that have not yet drifted cancel each other out. FLARE [5], FLAME [6], FL-MalDrift [7], DriftGuard [8], FedDAA [9], FedPLC [10], the label-free virtual-drift detection of FELK/FL-HVD [24], and Casado et al. [2] follow local detection with central decision: nine of the twelve concept-drift FL works we reviewed do so. The three exceptions are informative: FLASH detects drift server-side from the magnitude of aggregated parameter updates [23], Stallmann et al. use a global fuzzy-clustering detector [25], and Adaptive-FedAVG forgoes detection altogether, adapting the server-side learning rate to raise the plasticity of the learning phase, a passive alternative to local detection [27]. Our design follows the consensus; the specific signal, however, is label-free uncertainty (Section IV-A).

C. Label-free drift detection

The uncertainty drift detector (UDD) of Baier et al. [11] is the reference label-free method: for each input, the model performs T stochastic forwards with dropout enabled (MC dropout, [12]); the mean Shannon entropy feeds an ADWIN adaptive window test [13]. ADWIN is chosen over DDM/EDDM because these require binomial inputs, whereas the entropy signal is not restricted to a binomial distribution [11]. Lobo et al. [14] connect concept drift and model uncertainty, and uncertainty estimators are known to degrade under dataset shift—exactly the signal the detector exploits [15]. The published UDD uses $T = 50$ forwards; our $T = 5$ is a cost decision whose sensitivity is left as future work. D3M [16] is a complementary label-free method based on model disagreement, with false-positive guarantees under the covariate-shift assumption. PUDD [28] is complementary too: it tests the full distribution of a prediction-uncertainty index with a Pearson chi-square test whose significance level directly controls the false-positive rate, unlike UDD’s mean-tracking ADWIN stage.

D. Retraining orchestration and budgets

DriftGuard [8] formalizes the retraining decision as a tuple $\pi^t = (\text{Trig}, S, \theta)$: whether to retrain, which devices participate, and which parameters to update, reporting retraining-cost reductions of 80–83%. Modyn [20] separates trigger policies (time, volume, performance, drift) from data-selection policies; its drift trigger uses MMD (with KS among candidate statistics) and an adaptive percentile threshold. In the reviewed FL works, only DriftGuard [8] and ShiftEx [26] make the retraining budget an explicit parameter (a named quantity controlling the number of post-trigger training rounds). We adopt an explicit budget of a single global round and retrain on the distribution each client currently serves.

E. Metrics gap

The drift-detection benchmark of Cerqueira et al. [3] documents the lack of standardized temporal metrics (normalized detection delay, alarm rate, acceptance windows), as do FL drift surveys [19]. Detection latency [5], [11], retraining duration, and cost per retraining [8] are reported separately; none of the works we reviewed quantifies the time the served model stays below a quality threshold, the downtime metric we adopt, which mirrors the SLO practice of MLOps. Downtime language exists in adjacent ML-systems work—federated unlearning measures service downtime as the unavailability incurred while retraining to erase data [29]—but that is an availability notion (the service is offline), whereas we measure time served online yet below a quality threshold. We report the detection delay in raw virtual seconds; the normalized delay (NDT) defined by the benchmark [3] is left as future work.

III. Problem Statement

We consider synchronous FL with C clients and virtual time v , the abstract clock of our simulator. A model is

trained on clean data for R rounds, then enters production at time v_0 . Production data is served in monitoring ticks of Δt seconds, one unlabeled batch per client per tick.

Gradual drift model. A corruption applied to inputs X (labels preserved) is injected with a mixture fraction $f(v)$ that grows linearly from zero to one over N ticks:

$$f(v) = \min\left(1, \frac{v - v_0}{N \Delta t}\right), \quad v \geq v_0. \quad (1)$$

The ramp duration $N\Delta t$ is the drift width; its slope $1/N$ per tick is the independent variable of our study. Here $N \in \{5, 10, 20\}$ (50, 100, 200 s at $\Delta t = 10$ s).

Quality threshold and downtime. Let $a(v)$ be the accuracy of the served model on a test set mixed at the current fraction $f(v)$. The quality threshold τ defines the service state: the model is unavailable when $a(v) < \tau$. The downtime over a fixed horizon $[v_0, v_0 + H]$ is

$$D = \sum_{i=1}^m (v_{i+1} - v_i) \cdot \mathbf{1}[a(v_i) < \tau], \quad (2)$$

where $v_1, \dots, v_m$ are the evaluation instants and $v_{m+1} = v_0 + H$: the stepwise integral uses the last known quality, without interpolation, and includes the final interval up to the horizon (always an evaluation instant). The evaluation instants comprise tick boundaries, the retraining completion instant, and the horizon; the post-retraining grid is phase-shifted by the round duration.

Paired evaluation. The reactive policy is compared with a baseline that observes the same stream from the same checkpoint and never retrains. Both arms share the same random state, production start, and horizon H ; the baseline records the moment the policy would have triggered as a counterfactual. The comparison attributes the downtime difference to the retraining action within this paired design.

IV. Proposed Method

A. Local uncertainty detection (UDD per client)

Each client c runs a local detector over the shared global model: (1) the model is set to evaluation mode with dropout layers re-enabled; (2) $T = 5$ stochastic forwards produce class probabilities; (3) the mean Shannon entropy $H = -\sum_k p_k \log p_k$ over the batch is computed; (4) the scalar feeds a per-client ADWIN with $\delta = 0.002$ [11], [13]. Our ADWIN implementation only tests windows of at least 20 observations (the reference implementation [13] uses a bucket structure with smaller minima), because we feed one batch-mean entropy per tick per client rather than a per-instance stream. Each detector therefore receives 20 warm-up ticks on clean data before production starts. The warm-up calibrates the detectors but does not feed the collective policy. The detector never mutates the model: training modes are restored after each update, and only the inputs X are observed.

B. Collective quorum policy

At each production tick, the server receives the per-client alarm flags and maintains a sliding window of the last two ticks. The flagged fraction is the number of distinct clients flagged in the window divided by the number of clients. When the fraction reaches the quorum $\theta = 0.30$ (3 of 10 clients), the server triggers, and a latch prevents any further decision in the episode. In the baseline arm the same computation runs with `record_action=false` and no retraining happens. A single isolated alarm (a possible local false positive under continuous monitoring [18]) is not enough to trigger.

C. Reactive retraining on the current mixture

When the policy triggers at time t_d , each client's training dataset is replaced by a mixture with corrupted fraction $f(t_d)$, built with a deterministic per-client permutation, and a single synchronous FL round is executed with all clients. Each client holds 5,000 training images (an IID split of the 50,000-image training set), so the retraining set is the client's 5,000-image set mixed at $f(t_d)$. The retraining learns the current distribution and does not predict the future of the ramp. The budget is explicit (one round); if the required virtual time would exceed the remaining horizon, the decision is recorded and the action skipped (this did not occur in our grid). A retraining round consumes about 8 s of virtual time (one server timeout); in every detected episode the accuracy at the round's completion already meets τ , so the time-to-recovery equals the retraining duration.

D. Evaluation on the current mixture

The accuracy series is measured on the full test set (10,000 images) mixed at the current fraction $f(v)$: at production start the test is clean (fraction zero), and it becomes fully corrupted only when the ramp completes; a fully corrupted test would overstate the degradation during the ramp. Mixing uses a deterministic permutation seeded per episode, so both arms are evaluated on the identical tensor at every instant.

E. Severity calibration

Each corruption is calibrated on a clean checkpoint: all severities 1–5 are evaluated on the corrupted test set, and the smallest severity whose accuracy falls strictly inside (0.25, 0.50) is selected. The lower bound keeps the model well above chance (0.10 for ten classes), so recovery is achievable with a small budget; the upper bound guarantees the corruption pushes the service below τ (so downtime exists to be avoided).

V. Experimental Setup

A. Dataset and corruptions

We use CIFAR-10 with an IID partition across 10 clients and the full 10 classes in all phases. Four visual corruptions are implemented directly on tensors—Gaussian noise,

TABLE I
Severity calibration from the clean checkpoint (strict interval (0.25, 0.50)).

Corruption	Severity	Accuracy
Gaussian noise	3	0.3783
Frosted-glass blur	4	0.4022
Motion blur	1	0.3493
Fog	4	0.4875

TABLE II
Fixed configuration of every episode.

Parameter	Value
Dataset	CIFAR-10, IID, 10 clients
Initial rounds (clean)	20
Warm-up ticks (clean)	20
Tick duration	10 s
Batch size / epochs	32 / 1
Model	CNN 2.2M (4 conv, 2 fc)
Optimizer	Adam, lr 10^{-3}
Speed / timeout	uniform (p75; round ≈ 8 s)
Detector	ADWIN $\delta = 0.002$; MC dropout $T = 5$
Quorum / window	0.30 / 2 ticks
Retraining budget	1 round
Quality threshold τ	0.50
Production horizon	600 s
Seeds	42–46

frosted-glass blur, motion blur, and fog—with integer severities 1–5, per-seed generators, and preserved shapes and labels. They are inspired by, not a reproduction of, the CIFAR-10-C benchmark [22]; their internal parameters are project-specific: Gaussian noise adds perturbations with per-severity standard deviations of 0.08–0.24; frosted-glass blur permutes pixels within a 3×3 neighborhood and blends with a Gaussian blur (strength 0.2–1.0); motion blur convolves with a diagonal kernel of width 5–15; fog blends the image with a four-octave fractal noise map. The complete parameterization accompanies the released code.

B. Calibration result

The calibration, executed once from a clean checkpoint, selected the severities in Table I (procedure in Section IV-E).

C. Grid

The study grid is the Cartesian product of the four calibrated corruptions, five seeds (42–46), and three ramp durations (5, 10, 20 ticks), totaling 60 paired episodes with a fixed production horizon of $H = 600$ s. Table II lists the fixed configuration.

VI. Results

We report group means over the five seeds. We test two hypotheses: H1, the reactive policy reduces downtime relative to the no-retraining baseline; H2, the detection delay grows with the ramp duration. Given $n = 5$ per group, inferential claims are limited to a pooled paired

TABLE III
Detection delay (s) and mixture fraction at the trigger (group means over 5 seeds; detection rate per group; the failed Gaussian seed at 100 s is excluded from its delay mean).

Corruption	Delay (s)			Frac. at trigger	
	50 s	100 s	200 s	200 s	det.
Fog	100	112	138	0.69	5/5
Frosted-glass	100	108	136	0.68	5/5
Motion blur	98	112	140	0.70	5/5
Gaussian noise	164	158*	256	0.97	4/5*

Delay std across seeds: ≤ 6 s for the visual corruptions; 71–104 s for Gaussian noise. For the 50 and 100 s ramps the fraction at the trigger is 1.00 by construction (post-ramp triggers).

TABLE IV
Downtime (s): reactive arm versus paired no-retraining baseline (group means over 5 seeds; format: reactive / baseline).

Corruption	Ramp duration		
	50 s	100 s	200 s
Fog	58 / 550	24 / 504	0 / 408
Frosted-glass	68 / 560	40 / 524	3 / 454
Motion blur	68 / 562	50 / 530	14 / 466
Gaussian noise	130 / 558	178 / 526	112 / 448

Gaussian-noise dispersion is large (delay std up to 104 s; downtime std up to 184 s, range 38–540 s); visual corruptions have std ≤ 6 s (delay) and ≤ 10 s (downtime).

comparison; the per-group results are reported descriptively with dispersion. The primary measurements are the detection delay (production start to trigger), the mixture fraction at the trigger, and the downtime; secondary metrics are the clean retention (final clean accuracy minus pre-drift clean accuracy) and the final corrupted accuracy. Full per-seed values accompany the released artifacts.

A. Detection: 59 of 60 episodes

Detection succeeded in 59 of the 60 episodes (14 of 15 for Gaussian noise). The only failure is Gaussian noise, 100 s ramp, seed 42, analyzed in Section VII-A. Table III shows the detection delay per scenario and ramp. The delay grows with the ramp duration for the three visual corruptions, consistent with the ADWIN analysis of gradual change with constant slope [13]; Gaussian noise does not follow this trend and shows much larger dispersion (per-seed std of 71–104 s).

Fraction at the trigger. With ramps of 50 and 100 s the trigger occurs after the ramp completes (fraction 1.00) in every episode: the group-mean delay (98–164 s) exceeds the ramp duration. Only the 200 s ramp produces an intra-ramp trigger, and only for the three visual corruptions (fractions 0.68–0.70); Gaussian noise triggers at 0.97, just after the ramp ends.

B. Downtime: reactive policy dominates the baseline

Table IV reports the downtime of the reactive arm against the baseline: the reactive policy reduces the

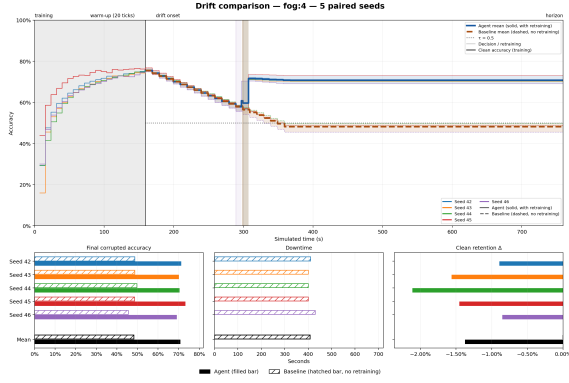


Fig. 1. Fog, 200s ramp, five paired seeds. Top: corrupted accuracy over simulated time (solid: reactive, dashed: baseline; thick lines: means); $\tau = 0.5$ dashed; shaded bands: the single retraining round. Bottom: final corrupted accuracy, downtime, and clean retention. The reactive arm never falls below τ (zero downtime).

mean downtime in all 12 groups, from 408–562s in the baseline to 0–178s in the reactive arm. A one-sided paired Wilcoxon signed-rank test rejects equal downtime between the arms ($W^+ = 1770$ over the 59 nonzero differences, the single tied pair excluded, $p = 1.2 \times 10^{-11}$); the pooled Hodges–Lehmann median downtime reduction is 462s ($\approx 95\%$ CI [450, 476]) with a paired Cliff’s delta of 0.98 (large) and 59 wins in 60 episodes. Given the paired design fixes identical pre-trigger trajectories, we treat the test as a sanity check of the average direction, the per-group contrasts remaining descriptive at $n = 5$: the Hodges–Lehmann reduction ranges 372–492s per group; Cliff’s delta is 1.00 (large) in 11 of 12 groups and 0.80 (large) for Gaussian noise at the 100s ramp (4 wins plus the single tie); and, since no exact 95% interval exists at $n = 5$ (smallest achievable tail mass 1/32), the exact intervals use the closest achievable coverage, 93.75% (87.5% for the tied Gaussian-noise group, where the tied pair is dropped, $m = 4$). In every pair the reactive arm’s downtime is at most the baseline’s (equal only in the failed episode). The fog scenario with the 200s ramp reaches zero downtime at $\tau = 0.5$ (Fig. 1): the trigger occurs at fraction 0.69 while the accuracy is still 0.568 (above τ), and the single round keeps it above τ (final corrupted accuracy 0.71).

C. Collective-quorum analysis

Whenever a decision is taken, all 10 clients raise an alarm at some point of the episode (59 of 59): the local detectors are individually sensitive to the gradual drift. The trigger fires exactly when the quorum is met at its minimum (0.30, three distinct clients in the two-tick window) in 21 of the 59 episodes; Gaussian noise is the extreme case, firing at the minimum in all detected episodes of the 100s ramp. The long ramp spreads the alarms in time: the flagged fraction at the decision drops with the ramp duration (fog 0.88→0.42, frosted 0.90→0.56, motion 0.66→0.54), because alarms arrive at more scattered ticks and co-occur later, at a smaller fraction of the ramp.

TABLE V

Clean retention Δ (reactive arm; group mean $\pm$ std over 5 seeds).

Corruption	Ramp duration		
	50 s	100 s	200 s
Fog	-0.096 ± 0.012	-0.094 ± 0.014	-0.014 ± 0.005
Frosted-glass	-0.152 ± 0.008	-0.156 ± 0.014	-0.022 ± 0.005
Motion blur	-0.199 ± 0.017	-0.208 ± 0.020	-0.016 ± 0.002
Gaussian noise	-0.084 ± 0.013	-0.066 ± 0.034	-0.072 ± 0.016

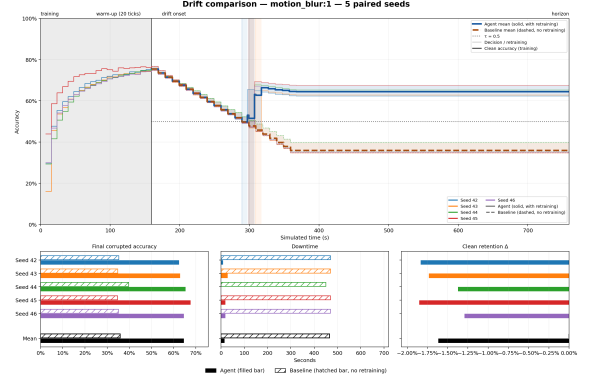


Fig. 2. Motion blur, 200s ramp, five paired seeds. The reactive arm recovers within one round and keeps the accuracy above τ for most of the horizon; the baseline keeps degrading to about 0.35.

These properties reflect the global, simultaneous drift of our scenario; staggered drifts, which motivate the quorum in [4], are not exercised here.

D. Clean retention and recovery behavior

Two findings deserve emphasis. First, long ramps preserve the clean-set accuracy (Table V): with the 200s ramp the retraining set keeps roughly 30% of clean data and the clean-accuracy loss drops to 1–2 points (versus 9–21 on short ramps) for fog, frosted-glass, and motion blur; Gaussian noise does not benefit: its trigger occurs after the ramp (fraction 0.97). Second, the recovery is not temporary: no episode shows the accuracy re-crossing τ within the fixed horizon; the retrained model generalizes to the fully corrupted regime. In 9 of the 20 episodes with the 200s ramp (all five fog runs, three frosted runs, one motion run) the accuracy never falls below τ . Group-mean final corrupted accuracy ranges from 0.60 to 0.71 for the reactive arm (excluding the failed episode at 0.33) and 0.35–0.48 for the baseline (Fig. 2).

VII. Discussion

Monitoring is not free, but it does not dominate the reactive budget: over an episode the $T = 5$ detector costs about 6.8 TFLOPs (4,000 batch-32 forwards, 50 per tick across the ten clients) against about 8.0 TFLOPs for the single retraining round, a ratio of 0.85–0.96 $\times$ depending on the accounting—the same order of magnitude, and roughly 47% of the reactive compute. On the measured device (Apple M5, PyTorch 2.12.1) one detector update takes

about 12.7ms per client; the per-tick monitor across the ten clients occupies about 0.13s, 1.3% of each 10s tick. The forward and FLOP counts refer to the virtual 80-tick episode; the 12.7ms figure is measured wall-clock. Had we kept the published UDD setting $T = 50$, monitoring would exceed the retraining round by roughly $9\times$, which is exactly why $T = 5$ was chosen as a cost decision; the accuracy sensitivity to T is left as future work.

The downtime conclusions are robust to a $10\times$ longer round: recomputing downtime post hoc from the recorded series—charging the pre-retrain accuracy over $[t_d, t_d + d]$, the first post-retrain record thereafter, with the controller’s budget rule applied—an 80s communication-dominated round still cuts the mean downtime from 507.5s (baseline) to 122.2s (76%); the per-group ranking, the zero-downtime fog case at the 200s ramp, and the detection-limited Gaussian behaviour are unchanged, the round’s extra cost being exactly the pre-retrain interval it extends (zero when the detector fires early). At $d = 800$ s the round no longer fits the 600s production horizon and the budget rule would skip retraining in all 59 triggered episodes, degenerating the agent to the baseline by policy rather than by training dynamics; the turning point is the smallest trigger-to-horizon slack, about 140s in this grid.

A. Gaussian noise is the limiting scenario

Gaussian noise is the only scenario with incomplete detection (14 of 15 episodes). With the 100s ramp and seed 42, the policy never triggers: only four of the ten clients raise an alarm during the whole episode, never three within the same two-tick window (maximum flagged fraction 0.20, at sparse ticks), so the quorum is never met; the other four seeds of the group detect with delays of 120–210s. Because the window and the quorum are identical across seeds, the failure is best explained as a per-client detector miss on this seed; our design cannot isolate an interaction among the signal, the window, and the quorum. Without a decision, the reactive arm degenerates exactly to the baseline (both arms end with the same downtime, 540s). With the 200s ramp the delays spread between 180 and 460s (std 104s), yet the reactive arm still cuts the baseline downtime by 75% (112 vs. 448s), consistent with the risk registered for ADWIN-style detectors under long, gentle drifts [18]. Fig. 3 shows the failed seed inside its group.

B. Why the long ramp behaves better

For the visual corruptions the 200s ramp concentrates three favorable effects. (i) The trigger occurs mid-ramp, at fractions 0.68–0.70, where the accuracy at the decision is 0.57 for fog (still above τ , hence zero downtime), 0.51 for frosted-glass, and 0.48 for motion blur. (ii) The retraining set mixes about 30% clean data, reducing catastrophic forgetting. (iii) The model retrained at fraction ≈ 0.7 generalizes to the fully corrupted regime, so the recovery holds until the horizon. Short ramps behave like the abrupt regime: detection completes after the ramp and

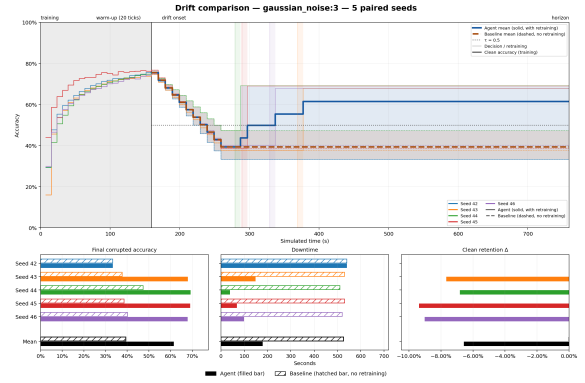


Fig. 3. Gaussian noise, 100s ramp, five paired seeds. Seed 42 never triggers: its trajectory (solid blue) does not recover, while the other four seeds recover to about 0.68. The group mean is pulled down by the failed seed.

the retraining set is almost fully corrupted. Gaussian noise triggers later (fraction 0.97) and does not enjoy effects (i) and (ii).

C. Limitations

Our results are bounded by the following declared limitations. (i) Five seeds per group limit the precision of the dispersion estimate. (ii) The model is a small CNN of about 2.2 million parameters; absolute accuracies under corruption are modest, as expected for small architectures [22]. (iii) The scenario assumes a single drift, one retraining round, an IID partition, uniform speed, and a fixed horizon; the paired comparison isolates the retraining action within these conditions. (iv) The fraction at the trigger is informative only in the long-ramp regime; the trigger occurs after the ramp completes for short ramps. (v) The published UDD uses $T = 50$ forwards; our $T = 5$ is a cost decision without published equivalence, and $\delta = 0.002$ is the fallback of the reference implementation rather than a calibrated value [11], [18]. (vi) Downtime is sampled at tick granularity plus the round-completion instant; the true crossing of τ can fall between ticks. (vii) Episodes ran on a CUDA GPU (PyTorch 2.5.1, CUDA 12.1) with non-deterministic algorithms; bit-identical reproduction was not confirmed, although each pair is validated to share an identical checkpoint, stream, and horizon. (viii) The severities were calibrated on a clean checkpoint from a separate training run; the calibration digest differs from the episode checkpoints, and severities are not re-validated per seed. (ix) The reported contrasts depend on $\tau = 0.5$ and the calibrated severities; we report descriptive group means over 12 groups without multiple-comparison correction. (x) The evaluation compares the reactive policy only with a no-retraining baseline; other comparators are future work, so our claims concern the value of retraining reactively rather than never, not the superiority of this detection mechanism over other triggers. (xi) No drift-free control episodes were run, so the detector’s false-

positive rate under continuous monitoring is unmeasured; sensitivity to τ , the quorum, and the window is future work.

VIII. Conclusion and Future Work

We studied reactive retraining in synchronous federated learning across the abrupt-to-gradual drift continuum, evaluating a reactive policy—per-client label-free UDD detection, a server-side quorum over a two-tick window, and a single retraining round on the current mixture—against a paired no-retraining baseline over 60 episodes with service downtime as the primary metric. Detection succeeded in 59 of 60 episodes; the delay grew with the ramp duration for the visual corruptions; the reactive arm reduced the mean downtime from 408–562s to 0–178s, reaching zero at $\tau = 0.5$ in the fog scenario; and long ramps preserved clean accuracy for the visual corruptions. Short ramps behaved like the abrupt regime. The single detection failure (Gaussian noise, one seed) is a per-client detector miss whose interaction with the window-quorum policy our design cannot isolate.

Future work includes: scheduled-retraining, first-alarm, centralized- detector, and oracle baselines to isolate the contribution of the detection mechanism; drift-free control episodes to measure false positives; a sensitivity sweep over τ and the calibration bounds; a quorum ablation, including staggered drift; more seeds and a formal dispersion analysis; recurrent drift; non-IID partitions and heterogeneous client speeds; incremental drift; larger models; and publishing the protocol as a reusable benchmark for FL-under-drift evaluation [3], [19].

Availability

The implementation and all artifacts—60 paired episodes, the aggregated summary, per-seed tables, the calibration table, and plots—are available at [repository URL – TODO].

References

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, PMLR 54, pp. 1273–1282, 2017.
- [2] F. E. Casado, D. Lema, M. F. Criado, R. Iglesias, C. V. Regueiro, and S. Barro, “Concept drift detection and adaptation for federated and continual learning,” *Multimedia Tools and Applications*, vol. 81, pp. 3397–3419, 2022.
- [3] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, “A framework for evaluating and benchmarking concept drift detection methods,” in *Proc. KDD*, 2026, doi: 10.1145/3770855.3819070.
- [4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, “Federated learning under distributed concept drift,” in *Proc. AISTATS*, PMLR 206, pp. 5834–5853, 2023.
- [5] T. Chow, U. Raza, I. Mavromatis, and A. Khan, “FLARE: Detection and mitigation of concept drift for federated learning based IoT deployments,” in *Proc. IEEE IWCMC*, pp. 989–995, 2023.
- [6] I. Mavromatis, S. De Feo, and A. Khan, “FLAME: Adaptive and reactive concept drift mitigation for federated learning deployments,” *arXiv:2410.01386*, 2024.
- [7] A. Patel, D. S. Tomar, R. K. Pateriya, and R. HariPriya, “FL-MalDrift: A federated learning framework for malware detection under local concept drift,” *Scientific Reports*, vol. 16, Art. 1821, 2026.
- [8] Y. Han, D. Wu, and B. Varghese, “DriftGuard: Mitigating asynchronous data drift in federated learning,” *arXiv:2603.18872*, 2026.
- [9] P. Fu and M. Tang, “FedDAA: Dynamic client clustering for concept drift adaptation in federated learning,” *arXiv:2506.21054*, 2025.
- [10] Q. Zhou et al., “FedPLC: Federated learning with dynamic cluster adaptation for concept drift on non-IID data,” *Sensors*, vol. 26, no. 1, Art. 283, 2026.
- [11] L. Baier, T. Schlör, J. Schöffner, and N. Kühn, “Detecting concept drift with neural network model uncertainty,” in *Proc. 56th Hawaii Int. Conf. on System Sciences (HICSS)*, pp. 835–844, 2023.
- [12] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in *Proc. ICML*, PMLR 48, pp. 1050–1059, 2016.
- [13] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SIAM Int. Conf. on Data Mining*, pp. 443–448, 2007.
- [14] J. L. Lobo, I. Laña, E. Osaba, and J. Del Ser, “On the connection between concept drift and uncertainty in industrial artificial intelligence,” in *Proc. IEEE CAI*, pp. 171–172, 2023.
- [15] Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” in *Proc. NeurIPS*, pp. 13991–14002, 2019.
- [16] V. Nguyen, C. Shui, V. Giri, S. Arya, A. Verma, F. Razak, and R. G. Krishnan, “Reliably detecting model failures in deployment without labels (D3M),” in *Proc. NeurIPS*, 2025.
- [17] O. Solozobov, “Label-free detection of governance evidence degradation in risk decision systems,” *arXiv:2604.17836*, 2026.
- [18] L. Poenaru-Olaru, L. Cruz, A. van Deursen, and J. S. Reller-meyer, “Are concept drift detectors reliable alarming systems?” *arXiv:2211.13098*, 2022.
- [19] O. A. Mahdi, E. Pardede, S. Bevinakoppa, and N. Ali, “Federated learning under concept drift: A systematic survey of foundations, innovations, and future research directions,” *Electronics*, vol. 14, no. 22, Art. 4480, 2025.
- [20] M. Böther et al., “Modyn: Data-centric machine learning pipeline orchestration,” *Proc. ACM on Management of Data*, vol. 3, no. 1, Art. 55, 2025.
- [21] M. F. Criado, F. E. Casado, R. Iglesias, C. V. Regueiro, and S. Barro, “Non-IID data and continual learning processes in federated learning: A long road ahead,” *Information Fusion*, vol. 88, pp. 263–280, 2022.
- [22] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *Proc. ICLR*, 2019.
- [23] K. Panchal et al., “Flash: Concept drift adaptation in federated learning,” in *Proc. ICML*, PMLR 202, pp. 26931–26962, 2023.
- [24] X. Chen et al., “Virtual concept drift detection and adaptation in federated data stream learning,” *Int. J. Data Science and Analytics*, vol. 21, Art. 46, 2026.
- [25] M. Stallmann, A. Wilbik, and G. Weiß, “Towards unsupervised sudden data drift detection in federated learning with fuzzy clustering,” in *Proc. IEEE FUZZ-IEEE*, 2024.
- [26] R. A. Bhope, K. R. Jayaram, P. Venkateswaran, and N. Venkatasubramanian, “Shift happens: Mixture of experts based continual adaptation in federated learning,” in *Proc. ACM Middleware*, pp. 455–468, 2025.
- [27] G. Canonaco, A. Bergamasco, A. Mongelluzzo, and M. Roveri, “Adaptive federated learning in presence of concept drift,” in *Proc. IEEE Int. Joint Conf. on Neural Networks (IJCNN)*, pp. 1–7, 2021.
- [28] P. Lu, J. Lu, A. Liu, and G. Zhang, “Early concept drift detection via prediction uncertainty,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 39, no. 18, pp. 19124–19132, 2025.
- [29] B. Zhang, H. Guan, H. K. Lee, R. Liu, J. Zou, and L. Xiong, “FedSGT: Exact federated unlearning via sequential group-based training,” *arXiv:2511.23393*, 2025.